Prince Sultan University
CCIS, Department of Computer Science
CS496 Emerging Topics: Quantum & Intelligent Computing
Blockchain Module
Smart Contracts Practice Sheet

What is a Smart Contract?

A **smart contract** is self-executing code stored on the Ethereum blockchain. Once deployed, it runs exactly as written — no third party can alter it, censor it, or shut it down. Think of it as a vending machine: you put in the right input (ETH + selection), and it automatically executes the programmed output (dispenses item + change), with no cashier needed.

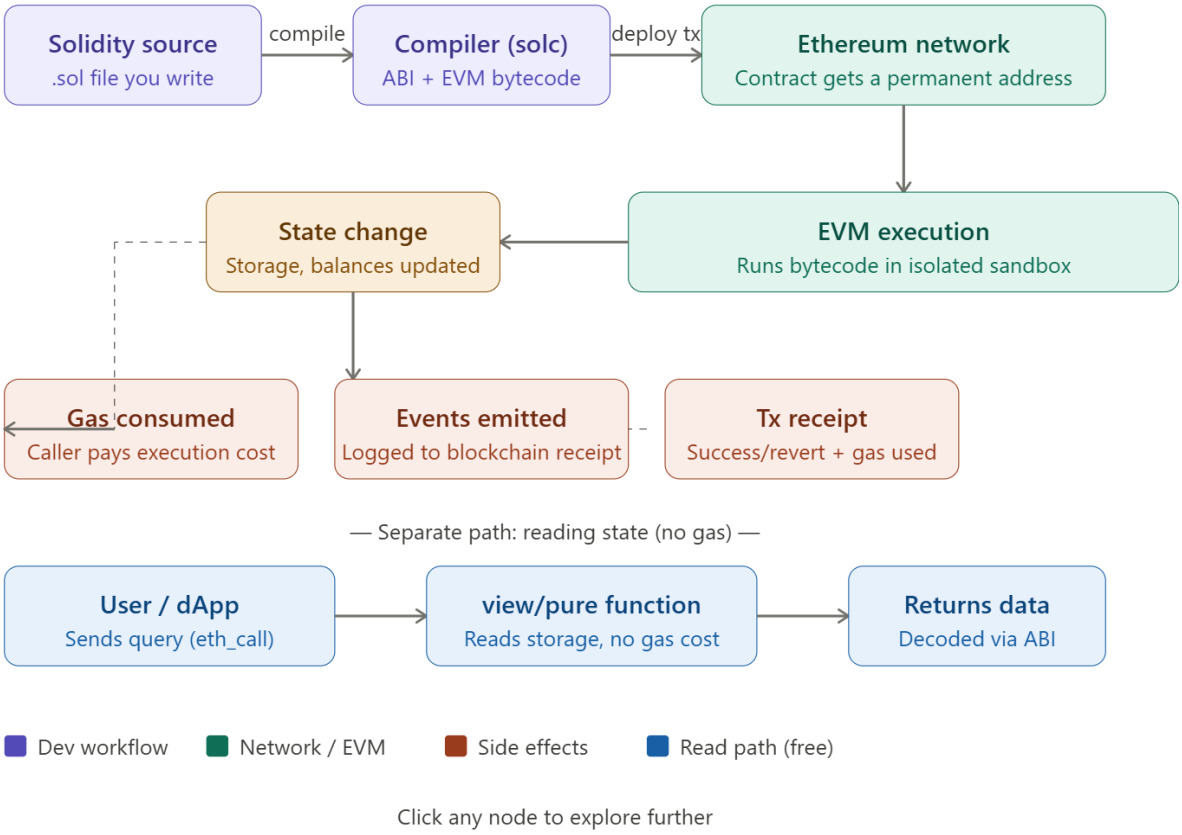
They're written in **Solidity** (most common), compiled to **EVM bytecode**, deployed to Ethereum's decentralized network, and then interacted with via transactions.

Another definition:

Smart contracts are self-executing, automated agreements written in code, removing intermediaries like banks or lawyers. They function like digital vending machines—if conditions (\(A\)) are met, then an action (\(B\)) occurs. They are ideal for automating payments, supply chain actions, escrow services, and insurance claims

The Architecture: How Execution Flows

The figure below shows the full lifecycle — from writing Solidity code to a transaction hitting the blockchain:



Scenario 1: A Simple Storage Contract

This is the "Hello World" of smart contracts. It stores and retrieves a number on-chain.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleStorage {
    uint256 private storedValue;   // Persisted in blockchain state

    event ValueChanged(address indexed by, uint256 newValue);

    function set(uint256 _value) public {
        storedValue = _value;
        emit ValueChanged(msg.sender, _value);
    }
```

```
    function get() public view returns (uint256) {
        return storedValue;  // FREE — no gas, reads from state trie
    }
}
```

Key concepts here:

- uint256 — unsigned 256-bit integer (Ethereum's native word size)
- public — accessible from outside the contract
- view — read-only, no state change, no gas cost when called externally
- emit ValueChanged(...) — logs an event to the transaction receipt, readable by front-ends
- msg.sender — the Ethereum address that called this function

Deploying on Remix IDE (the real platform):

1. Go to remix.ethereum.org
2. Paste the code, select compiler 0.8.20+
3. Deploy to **Remix VM (Shanghai)** — instant in-browser blockchain
4. Call set(42) → costs gas (writes state)
5. Call get() → returns 42, costs no gas

Scenario 2: ERC-20 Token Contract

This is one of the most important real-world use cases — creating your own fungible token (like USDC or DAI).

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// OpenZeppelin's battle-tested base gives us transfer, approve, allowance
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract MyToken is ERC20, Ownable {
    uint256 public constant MAX_SUPPLY = 1_000_000 * 10**18; // 1M tokens
```

```
    constructor(address initialOwner)
        ERC20("MyToken", "MTK")
        Ownable(initialOwner)
    {
        // Mint 100,000 tokens to the deployer at launch
        _mint(initialOwner, 100_000 * 10**18);
    }

    // Only owner can mint more (up to MAX_SUPPLY)
    function mint(address to, uint256 amount) public onlyOwner {
        require(totalSupply() + amount <= MAX_SUPPLY, "Exceeds max supply");
        _mint(to, amount);
    }
}
```

Other Simple Examples:

Voting:

```
pragma solidity ^0.8.0;

contract VotingSystem {
    uint public candidateA;
    uint public candidateB;

    function voteCandidateA() public {
        candidateA += 1;
    }

    function voteCandidateB() public {
        candidateB += 1;
    }

    function getVotes() public view returns (uint, uint) {
        return (candidateA, candidateB);
    }
}
```

Transferring a coin:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract SimpleTokenTransfer {
    mapping(address => uint) public balances;

    function mint(uint amount) public {
        balances[msg.sender] += amount;
    }

    function transfer(address recipient, uint amount) public {
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
        balances[recipient] += amount;
    }
}
```

Banking Example:

```
pragma solidity ^0.8.0;

contract SimpleBank {
    mapping(address => uint) balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint amount) public {
        require(balances[msg.sender] >= amount, "Insufficient funds");
        balances[msg.sender] -= amount;
        payable(msg.sender).transfer(amount);
    }

    function checkBalance() public view returns (uint) {
        return balances[msg.sender];
    }
}
```

Simulating Vending Machines:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.7;
```

```
contract VendingMachine {
    address public owner;
    mapping (address => uint) public cupcakeBalances;

    constructor() {
        owner = msg.sender;
        cupcakeBalances[address(this)] = 100; // Machine starts with 100 items
    }

    function purchase(uint amount) public payable {
        require(msg.value >= amount * 1 ether, "Pay 1 ETH per cupcake");
        require(cupcakeBalances[address(this)] >= amount, "Not enough stock");
        cupcakeBalances[address(this)] -= amount;
        cupcakeBalances[msg.sender] += amount;
    }
}
```